

PINBALL HETIC — WEB3

Les tests logiciels : théorie et bilan du projet

Support de préparation à l'oral — cours général sur les tests logiciels, puis état des lieux complet et chiffré des tests écrits sur le projet Pinball Hetic.

Partie 1 — Cours général sur les tests

Partie 2 — Bilan des tests du projet

Document préparé le 8 juillet 2026 — Imrane Mesbah

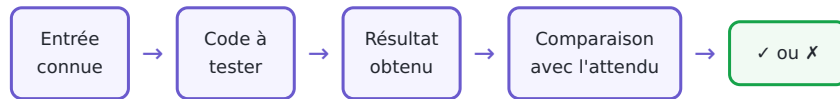
01

Cours général — Qu'est-ce qu'un test logiciel ?

Définitions, familles de tests, exemples simples puis exemples "pro" réutilisables à l'oral.

1. Qu'est-ce qu'un test logiciel ?

Un **test logiciel** est un morceau de code dont le seul but est de vérifier qu'un autre morceau de code se comporte comme prévu. Concrètement : on donne une entrée connue à une fonction, on compare ce qu'elle renvoie à ce qu'on *attend*, et le test répond par un verdict binaire — **succès (✓)** ou **échec (X)**. Un test s'exécute automatiquement (pas besoin d'un humain qui clique), en quelques millisecondes à quelques secondes, et peut être rejoué à l'infini, à chaque modification du code.



Exemple le plus simple possible

```
function addition(a, b) {  
  return a + b;  
}  
  
// Le test :  
test("addition(2, 3) doit valoir 5", () => {  
  expect(addition(2, 3)).toBe(5);  
});
```

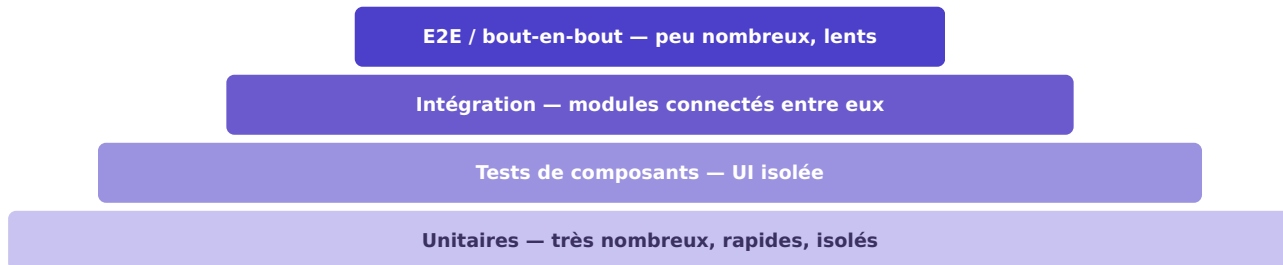
Si un jour quelqu'un modifie `addition` par erreur (par exemple `a - b` au lieu de `a + b`), ce test échoue immédiatement et prévient l'équipe — avant que le bug n'atteigne un utilisateur.

POURQUOI ON TESTE

Détecter les régressions (un changement qui casse quelque chose d'autre), pouvoir refactorer du code en confiance, documenter le comportement attendu, et bloquer automatiquement une mise en production si quelque chose est cassé (porte de qualité en CI).

2. Les grandes familles de tests

On classe généralement les tests selon leur **périmètre** : est-ce qu'ils testent une seule fonction isolée, plusieurs modules connectés entre eux, ou l'application entière comme le ferait un utilisateur réel ? On représente souvent ça par une **pyramide de tests** : plus on monte, plus les tests sont réalistes mais lents, coûteux et fragiles ; plus on descend, plus ils sont rapides, nombreux et faciles à maintenir.



La pyramide des tests : on construit une large base de tests unitaires rapides, et on réserve les tests bout-en-bout à quelques parcours critiques.

2.1 — Test unitaire (unit test)

Teste **une seule fonction ou classe, isolée** de ses dépendances (pas de réseau, pas de base de données, pas d'interface). C'est le type de test le plus rapide et le plus nombreux dans un projet sain.

EXEMPLE SIMPLE

Tester qu'une fonction `estMajeur(age)` renvoie `true` pour 18 et `false` pour 17.

EXEMPLE "PRO" POUR L'ORAL

Dans une app e-commerce, tester en isolation la fonction qui calcule le prix final d'un panier (remise + TVA + frais de port), sans toucher à la vraie base de données ni au vrai service de paiement.

2.2 — Test d'intégration

Vérifie que **plusieurs unités fonctionnent correctement ensemble** : une fonction qui appelle une vraie base de données de test, ou deux modules internes qui communiquent via des événements. Plus lent qu'un test unitaire (il y a de vraies I/O), mais détecte des bugs qu'aucun test isolé ne peut voir (mauvais format d'échange entre deux modules, par exemple).

EXEMPLE SIMPLE

Tester qu'un formulaire d'inscription écrit bien un nouvel utilisateur dans une base de données de test, avec le bon mot de passe haché.

EXEMPLE "PRO" POUR L'ORAL

Vérifier que le service de commande appelle correctement l'API sandbox d'un prestataire de paiement (Stripe, PayPal...) et met à jour le statut de la commande en base selon la réponse reçue.

2.3 — Test end-to-end (E2E) / système

Simule un **utilisateur réel** qui utilise l'application du début à la fin, en général via un vrai navigateur piloté automatiquement (Playwright, Cypress, Selenium). C'est le plus réaliste, mais aussi le plus lent et le plus fragile (un simple changement visuel peut le casser) — on en garde donc peu, sur les parcours vraiment critiques.

EXEMPLE SIMPLE

Un test qui ouvre la page d'accueil dans un navigateur automatisé et vérifie que le titre s'affiche bien à l'écran.

EXEMPLE "PRO" POUR L'ORAL

Un scénario qui se connecte à l'appli, ajoute un article au panier, va jusqu'au paiement et vérifie que la commande apparaît bien dans l'historique — le parcours qui fait gagner de l'argent à l'entreprise, donc celui qu'on ne peut pas se permettre de casser.

2.4 — Autres types utiles à connaître

- **Test de non-régression** : rejoue un scénario qui a déjà causé un bug par le passé, pour être sûr qu'il ne revienne jamais.
- **Smoke test** : vérification ultra-rapide après un déploiement — "l'appli démarre et répond", sans détail. Sert d'alarme incendie, pas de diagnostic complet.
- **Test de performance / charge** : mesure le temps de réponse ou le comportement sous forte charge (ex. 10 000 utilisateurs simultanés).
- **Test manuel / exploratoire** : un humain utilise l'appli sans script prédéfini pour trouver des problèmes inattendus — complémentaire, pas automatisable.

3. Vocabulaire indispensable pour l'oral

TDD

TEST-DRIVEN DEVELOPMENT : ÉCRIRE LE TEST AVANT LE CODE
(RED → GREEN → REFACTOR)

Mock / Stub

REEMPLACER UNE VRAIE DÉPENDANCE (API, DB) PAR UNE FAUSSE,
POUR ISOLER CE QU'ON TESTE

Couverture

% DU CODE RÉELLEMENT EXÉCUTÉ PAR LES TESTS — UN
INDICATEUR, PAS UNE GARANTIE DE QUALITÉ

Assertion

LA LIGNE QUI COMPARE LE RÉSULTAT OBTENU À CE QUI EST
ATTENDU (`EXPECT(...).TOBE(...)`)

Fixture

DONNÉE OU ÉTAT PRÉPARÉ À L'AVANCE POUR QU'UN TEST PARTE
D'UN CONTEXTE CONNU

CI Gate

LES TESTS TOURNENT AUTOMATIQUEMENT À CHAQUE PUSH ; LE
MERGE EST BLOQUÉ S'ILS ÉCHOUENT

NUANCE À RESSORTIR À L'ORAL

100% de couverture ne veut pas dire "0 bug" : la couverture mesure si une ligne a été *exécutée* par un test, pas si son résultat a vraiment été *vérifié* correctement. On peut exécuter une ligne sans jamais tester le bon cas.

Exemple "pro" complet à raconter à l'oral

Un pattern courant en entreprise : la CI exécute les tests **par étages, du plus rapide au plus lent**. D'abord les tests unitaires (quelques secondes, des milliers de cas) ; s'ils passent, les tests d'intégration (connectés à une vraie base de test) ; s'ils passent, un petit nombre de tests E2E critiques (connexion, paiement). Si un étage échoue, les suivants ne se lancent même pas — ça fait gagner du temps de CI et donne un retour rapide au développeur. C'est exactement la logique de la pyramide de tests appliquée en pratique.

02

Bilan des tests sur Pinball Hetic

Stack utilisée, inventaire complet, et couverture de code mesurée à jour (8 juillet 2026).

4. La stack de test du projet

Le projet n'utilise **ni Jest ni Vitest** : les tests tournent avec **bun test**, le test-runner intégré au runtime Bun (syntaxe proche de Jest : `describe / test / expect`, importés depuis `bun:test`). Les composants React (dmd, backglass) sont testés dans un DOM simulé (**happy-dom**) via `@testing-library/react`. Au sommet de la pyramide, **3 specs Playwright** couvrent un parcours "smoke" par écran, mais nécessitent la stack Docker démarrée en vrai — elles ne font **pas** partie du pipeline CI automatique actuel (`.github/workflows/ci.yml` ne lance que lint/build/test unitaires).

Convention du repo : chaque fichier source a son test **juste à côté** (`Plunger.ts` + `Plunger.test.ts`), pas de dossier `__tests__` / séparé.

5. Inventaire chiffré — vue d'ensemble

171 FICHIERS DE TEST (.TEST.TS / .TEST.TSX)	1 878 TESTS UNITAIRES/COMPOSANTS AU TOTAL, TOUS VERTS
3 SPECS E2E PLAYWRIGHT (SMOKE, HORS CI AUTO)	0 TEST EN ÉCHEC SUR L'ENSEMBLE DU REPO

Répartition par type de test

TYPE	FICHIERS	CE QUE ÇA TESTE
Tests unitaires purs (<code>.test.ts</code>)	158	Logique métier / physique isolée, sans DOM ni React (game-engine, maps, server...)
Tests de composants React (<code>.test.tsx</code>)	13	Composants d'affichage (dmd, backglass, ui) rendus dans happy-dom
Tests end-to-end (Playwright <code>.spec.ts</code>)	3	Smoke test par écran, contre la stack Docker réelle

DEUX CHIFFRES À DISTINGUER À L'ORAL

Sur les 171 fichiers de test écrits, **tous ne sont pas exécutés par la CI aujourd'hui**. Le script `test` de `apps/playfield` est limité à `bun test src/audio` au lieu de `bun test` — 17 fichiers de test sur 18 dans `playfield` existent, passent, mais ne sont jamais lancés par `bun run test` à la racine ni par la CI GitHub Actions. Résultat : la CI exécute réellement **1 712 tests sur 154 fichiers**, contre 1 878 tests sur 171 fichiers si tout était branché.

6. Couverture de code — détail par package

Mesuré le 8 juillet 2026 avec `bun test --coverage`, en exécutant l'intégralité des tests de chaque package (pas seulement ce que la CI lance aujourd'hui pour `playfield`, afin d'avoir la mesure la plus honnête possible).

PACKAGE / APP	FICHIERS	TESTS	% LIGNES COUVERTES
packages/shared-types	2	28	100.00%
packages/ui	1	6	100.00%
packages/dmd-core	10	191	96.82%
apps/backglass	17	150	97.96%
apps/dmd	4	28	92.44%
apps/server	6	73	94.49%
apps/input-bridge	4	43	87.50%
packages/game-engine	82	928	76.15%
packages/maps (registry + strangerthings + zelda)	27	253	40.86%
↳ dont maps/strangerthings	18	151	46.83%
↳ dont maps/zelda	6	77	26.85%
apps/playfield (suite complète, 18 fichiers)	18	178	26.77%
↳ apps/playfield tel que lancé par la CI aujourd'hui (1 fichier)	1	12	20.40%

COUVERTURE GLOBALE DU PROJET — LA VRAIE MESURE

En pondérant par le nombre de lignes de chaque package (et pas en faisant une simple moyenne des pourcentages), la couverture **réelle** du repo est de **47,61%** des lignes si `playfield` tournait entièrement, et **46,92%** avec le script CI limité actuel (13 164 lignes couvertes sur 28 057). Total repo : 29 137 lignes de code testables, 13 873 réellement exercées par un test.

PIÈGE À CONNAÎTRE POUR L'ORAL

Le tableau récapitulatif que `bun test --coverage` affiche dans le terminal calcule en réalité une **moyenne non pondérée des pourcentages par fichier**, pas un vrai ratio global lignes-couvertes / lignes-totales. Exemple concret : pour `game-engine`, le terminal affiche 92,14% (moyenne des % de chaque fichier), alors que le vrai ratio pondéré en lignes est de 76,15% — parce que beaucoup de petits fichiers à 100% tirent la moyenne vers le haut, alors que quelques gros fichiers peu couverts pèsent plus lourd dans le total réel. Toujours préciser la méthode de calcul quand on annonce un chiffre de couverture.

7. Points d'attention à mentionner à l'oral

- Le script `test de playfield` ne couvre que `src/audio` (1 fichier / 18) — 17 fichiers de test bien réels et fonctionnels sont invisibles pour la CI et pour toute mesure de couverture basée sur ce qui tourne officiellement.
- La map **Zelda** est nettement moins couverte que **Stranger Things** (26,85% contre 46,83% en lignes) — cohérent avec le fait que c'est la map la plus récente et que la dette technique identifiée après sa livraison (duplication de code) est encore partiellement en cours de résorption.
- Les tests **E2E Playwright** ne sont pas exécutés automatiquement par la CI GitHub Actions actuelle — seule la commande `bun run test:e2e`, lancée manuellement contre une stack Docker démarrée, les exécute.
- **0 test en échec** sur l'ensemble des 1 878 tests exécutés — la base est saine, le sujet n'est pas la fiabilité des tests existants mais leur **périmètre** (parties du code encore non testées, surtout côté 3D/orchestration).